

Tecnología Digital VI - Guía de Ejercicios 1

Juan Ignacio Elosegui

Septiembre 2025

Ejercicio 1

Inciso A

Un **modelo determinístico** es aquel que, dado un único training set fijo, siempre produce el mismo resultado (las mismas predicciones o los mismos parámetros). Lo contrario sería un **modelo estocástico**.

Recordar que *leave-one-out-cross-validation* es un método de validación de modelos que entrena con todas las observaciones del *training set* y prueba con un único dato aislado a modo de validación. Se hace n veces y se promedian los resultados. Es bueno porque entrena y prueba con todos los datos, pero, es caro computacionalmente si tenés varias observaciones (tendrías que repetir el proceso muchas veces).

Podemos tener una iteración n_1 de LOOCV y tener una performance de 0,5, luego tener en la iteración n_2 una performance de 0.99. Las iteraciones de LOOCV pueden llegar a ser (cosa que es muy poco probable) todas iguales o, más probable, todas distintas. Pero, en definitiva, se va a llegar siempre al mismo promedio dado que es un modelo determinístico y contamos con un training set fijo.

Conclusión: es **falso**. Siempre se consigue el mismo resultado exclusivamente en un modelo determinístico, no en uno estocástico.

Inciso B

Recordar que una matriz densa guarda todos sus elementos explícitamente, aunque muchos sean ceros. Una **matriz rala o esparsa** tiene la mayoría de sus elementos en cero.

En una matriz densa, digamos de 1000×1000 , hay que guardar 1.000.000 de números. En una matriz rala se usa un formato especial: como la mayoría son ceros, directamente nos interesan los elementos de la matriz que NO son cero. Es más eficiente de buscar y de almacenar, por lo que termina usando muchísima menos RAM, pero sólo en el caso que sea muy esparsa la matriz. Ya llega un punto que ni conviene hacer esto.

Conclusión: es **falso**. Si la matriz esparsa es bien esparsa, sí conviene, porque ahorra RAM y tiempo, pero no siempre. Si la matriz es más o menos densa, no aplica el formato ralo porque puede llegar a consumir más RAM y tiempo de lo normal. En definitiva, depende qué tan esparsa sea para que adoptemos ese formato.

Inciso C

Con los clasificadores que usan probabilidades (como regresión logística o un árbol de decisiones) se usa el *threshold*, que es básicamente un umbral de corte:

- Si $\hat{p}(Y = 1|X) \geq \text{threshold}$ entonces predigo positivo.
- Si $\hat{p}(Y = 1|X) < \text{threshold}$ entonces predigo negativo.

Subir el *threshold* hará que el clasificador sea más exigente a la hora de predecir una observación como positivo. Vamos a tener menos positivos predichos y además se nos van a escapar los positivos verdaderos. Si bajamos el *threshold*, no se nos van a escapar tan fácil los positivos verdaderos.

Recordar que el *recall* es la proporción de positivos reales que atrapa nuestro modelo:

$$\text{recall} = \frac{TP}{TP + FN}$$

Es decir, de todos los casos que sí eran positivos, ¿cuántos marcó como positivos el modelo?

En definitiva, si bajamos el *threshold*, tendremos más positivos verdaderos pero también a costa de tener más falsos positivos. El *recall* subirá en este caso. Si subimos el *threshold*, tenemos menos positivos predichos, así que por más que sean positivos verdaderos o falsos, el valor de *recall* bajará.

Conclusión: es **falso**. Aumentar el *threshold* significa disminuir el *recall*.

Inciso D

El valor de F_1 está dado por:

$$F_1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

y lo ideal sería que este valor sea lo más alto posible, ya que tiene una forma de Cobb-Douglas (más es mejor).

3 Si el score F_1 dio como valor 0.3, se puede despejar de la siguiente manera la ecuación:

$$\begin{aligned} F_1 &= 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \\ 0.3 &= 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \\ 0.3 \cdot (\text{precision} + \text{recall}) &= 2 \cdot (\text{precision} \cdot \text{recall}) \\ 0.15 \cdot (\text{precision} + \text{recall}) &= \text{precision} \cdot \text{recall} \\ 0.15 \cdot \text{precision} + 0.15 \cdot \text{recall} &= \text{precision} \cdot \text{recall} \\ \text{precision} \cdot \text{recall} - 0.15 \cdot \text{precision} &= 0.15 \cdot \text{recall} \\ \text{precision}(\text{recall} - 0.15) &= 0.15 \cdot \text{recall} \\ \text{precision} &= \frac{0.15 \cdot \text{recall}}{\text{recall} - 0.15} \end{aligned}$$

Tendremos como resultado pares de la forma $(\text{precision}, \text{recall})$ que den como resultado el score $F_1 = 0.3$. Notar que $\text{recall} \neq 0.15$. Probemos valores para verificar si efectivamente los dos valores deben ser bajos para que el score sea bajo.

- $(precision, recall) = (precision, 0.8) \Rightarrow (0.18, 0.8)$ Precision bajo, Recall alto.
- $(precision, recall) = (precision, 0.5) \Rightarrow (0.21, 0.5)$ Precision bajo, Recall intermedio.
- $(precision, recall) = (precision, 0.16) \Rightarrow (2.4, 0.16)$???
- $(precision, recall) = (precision, 0.3) \Rightarrow (0.3, 0.3)$ Precision y Recall bajos.

Conclusión: el enunciado es **falso**. El primer ítem de la demostración es un contraejemplo: el *recall* es alto, pero el *precision* es bajo.

Inciso E

Conclusión: el enunciado es **verdadero**. La métrica *precision* nos dice cuántos positivos verdaderos conseguimos de todos los positivos verdaderos y falsos positivos que clasificamos.

$$precision = \frac{TP}{TP + FP}$$

Ejercicio 2

Inciso A

El objetivo es validar un modelo y estimar cómo rendirá ante un potencial *test set*. La idea es separar los datos de entrenamiento y validar en una observación o un subconjunto de observaciones distinto. Es una simulación de *test sets*.

Inciso B

1) LOOCV 2) k-fold cross-validation y 3) holdout set.

- LOOCV es más completo, pero si quieres validar todo el dataset con la única observación que quedó afuera, entonces deberías repetir el proceso de entrenamiento y validación tantas veces como tengas observaciones.
- k-fold cross-validation con $k = 3$ implica dividir el dataset en 3 subconjuntos. Esto quiere decir, un subconjunto sirve como *validation set*, y el resto como entrenamiento. Esto se debe repetir para todos los 3 subconjuntos. Es caro porque estamos hablando de subconjuntos grandes.
- Holdout set es el más eficiente solamente porque es un k-fold cross-validation con dos *folds*. Una mitad para entrenamiento, mientras que la otra mitad es para validación.

Inciso C

Usaría k-fold cross-validation si quiero hacer una validación muy bien estimada de mi modelo, ya que puedo hacer más de dos *folds* (como se usan dos o tres en *holdout set*). Más *folds* implica tener una mejor estimación de la performance de mi modelo.

Usaría *holdout set* si no quisiera tener una estimación tan buena y estuviera corto de capacidad de cómputo en la máquina.

Inciso D

Inciso E